

Desenvolvimento de Algoritmos para Pré-processamento de Dados

João Vítor da C. de Almeida ¹, André Almeida Gomes Neto², Deivid Souza dos Santos Oliveira³

¹Instituição - Centro Universitário de Excelência/Sistemas de Informação (UNEX)

²Departamento de Sistemas e Computação
Centro Universitário de Excelência(UNEX) – Feira de Santana, BA – Brasil

Abstract. *Data preprocessing is an essential step in the development of machine learning models, responsible for ensuring the quality, consistency, and integrity of the data used to train the algorithms. This work presents the implementation of a proprietary data preprocessing library in Python, developed with native Python features. The proposal aims to transform raw and inconsistent data into a clean, analysis-friendly format, ensuring the reliability of the results obtained.*

Resumo. *O pré-processamento de dados é uma etapa essencial no desenvolvimento de modelos de aprendizado de máquina, responsável por garantir a qualidade, consistência e integridade dos dados utilizados no treinamento dos algoritmos. Este trabalho apresenta a implementação de uma biblioteca própria de pré-processamento de dados em Python, desenvolvida com recursos nativos da linguagem. A proposta visa transformar dados brutos e inconsistentes em um formato limpo e adequado para análise, assegurando a confiabilidade dos resultados obtidos.*

1. Introdução

Modelos de recomendação dependem fortemente da qualidade dos dados que os alimentam. Dados inconsistentes, duplicados ou em escalas diferentes podem comprometer a precisão das previsões. O pré-processamento, portanto, é um processo de extrema importância no conjunto de etapas para o aprendizado de máquina. Este relatório descreve os desafios identificados no conjunto de dados do projeto e as soluções propostas para garantir a integridade e a representatividade das informações.

2. Fundamentação Teórica

O pré-processamento de dados é uma etapa indispensável em projetos de aprendizado de máquina. Ele garante que os dados utilizados sejam consistentes, íntegros e adequados para alimentar algoritmos, evitando vieses e distorções que poderiam comprometer a

qualidade dos modelos. A seguir, detalham-se os principais conceitos que embasam a implementação da biblioteca proposta.

A implementação baseia-se em conceitos centrais do pré-processamento de dados:

2.1 Tratamento de Valores Ausentes

Valores ausentes (NaN ou None) são comuns em bases reais e podem surgir por falhas de registro, erros humanos ou respostas incompletas. Sua presença pode distorcer estatísticas básicas e reduzir a performance de modelos preditivos. As principais estratégias incluem:

- **Remoção de registros:** adequada quando a quantidade de dados faltantes é pequena. Exemplo: excluir 2% de linhas com valores nulos em uma base de 10.000 registros não comprometeria toda a representatividade.
- **Imputação estatística:** substituição por média, mediana ou moda. Exemplo: preencher valores ausentes de “tempo de entrega” com a mediana evita distorções causadas por outliers.
- **Modelos de imputação:** podem prever valores ausentes com base em padrões observados.
- **Categoria especial “faltante”:** em variáveis categóricas, a ausência pode ser informativa. Exemplo: um cliente que não avaliou um restaurante pode indicar indiferença ou insatisfação.

2.2 Escalonamento de Dados

Diferenças de escala entre variáveis podem introduzir vieses. Por exemplo, se o valor de um pedido varia de 0 a 500 e a nota de avaliação de 1 a 5, tenderá a dar mais peso ao valor monetário.

- **Min-Max Scaling:** normaliza os valores para o intervalo $[0,1]$. Exemplo: transformar valores de pedidos de R\$50 a R\$200 em uma escala proporcional entre 0 e 1.
- **Standard Scaling (Z-score):** centraliza os dados em média 0 e desvio padrão 1. Exemplo: notas de restaurantes (1 a 5) passam a ser representadas em desvios-padrão em relação à média, facilitando comparações..

2.3 Codificação de Variáveis Categóricas

Variáveis categóricas, como “tipo de cozinha” ou “forma de pagamento”, carregam informações valiosas, mas precisam ser convertidas em números para que algoritmos possam processá-las.

- **Label Encoding:** Converte categorias em números inteiros. Útil para modelos que interpretam ordens.

Exemplo: “Italiana”=0, “Japonesa”=1, “Brasileira”=2. É eficiente, mas pode induzir uma ordem inexistente.

- **One-Hot Encoding:** Cria colunas binárias para cada categoria. Evita atribuição de ordem implícita.

Exemplo: “cozinha_italiana”, “cozinha_japonesa”, “cozinha_brasileira”. Evita a falsa ordem, mas pode aumentar a dimensionalidade em bases com muitas categorias.

2.4 Validação da Estrutura do Data

Outro aspecto essencial é garantir que todas as colunas possuam o mesmo número de registros. Caso contrário, operações de análise ou treinamento podem falhar. Exemplo: se a coluna “valor_pedido” tiver 100 linhas e a coluna “nota_restaurante” tiver 98, o dataset estará inconsistente, comprometendo qualquer modelagem.

3. Metodologia

A biblioteca foi desenvolvida em Python puro, utilizando apenas estruturas nativas como listas, dicionários e conjuntos. O dataset é representado como um **dicionário**, em que cada chave corresponde a uma coluna e cada valor é uma lista representando as linhas.

3.1 Estrutura da Biblioteca

- **Classe Preprocessing:** encapsula a API principal, permitindo encadeamento de métodos.
- **Classe MissingValueProcessor:** responsável por lidar com valores ausentes. Métodos:

- `isna()`: identifica linhas com valores nulos.
- `notna()`: identifica linhas sem valores nulos.
- `fillna()`: preenche valores ausentes com média, mediana, moda ou valor fixo.
- `dropna()`: remove linhas com valores ausentes.
- **Classe Scaler**: responsável pelo escalonamento de variáveis numéricas. Métodos:
 - `MinMax_scaler()`: normaliza valores para o intervalo $[0,1]$.
 - `standard_scaler()`: padroniza valores para média 0 e desvio padrão 1.
- **Classe Encoder**: responsável pela codificação de variáveis categóricas. Métodos:
 - `label_encode()`: atribui inteiros únicos a cada categoria.
 - `oneHot_encode()`: cria colunas binárias para cada categoria.

4. Resultados e discussão

Foi implementada uma arquitetura modular para o pré-processamento de dados, contemplando operações fundamentais como tratamento de valores ausentes, escalonamento de variáveis numéricas e codificação de variáveis categóricas. A seguir, discutem-se os principais resultados e implicações de cada componente.

4.1 Tratamento de valores ausentes

A classe **MissingValueProcessor** oferece métodos robustos para identificar (`isna`, `notna`) remover (`dropna`) e imputar (`fillna`) valores nulos

Resultado: A aplicação desses métodos garante maior integridade ao conjunto de dados, reduzindo o impacto de valores faltantes sobre modelos preditivos. A escolha do método de imputação deve considerar a distribuição da variável e o contexto analítico.

4.2 Escalonamento de Variáveis numéricas

A classe **Scaler** implementa dois métodos de normalização:

- transforma os dados para o intervalo $[0, 1]$, útil para algoritmos sensíveis à magnitude.
- centraliza os dados com média zero e desvio padrão unitário, ideal para modelos que assumem distribuição normal.

Resultado: O escalonamento melhora a performance de algoritmos, além de acelerar a convergência em processos de otimização.

4.3 Integração e Orquestração

A **classe Encoder** atua como fachada unificadora, garantindo consistência na aplicação das transformações e validação da estrutura do data-set. Essa abordagem modular facilita a reutilização, manutenção e extensão do código.

Discussão: A separação de responsabilidades entre classes promove clareza e escalabilidade. No entanto, a ausência de tratamento automático para tipos de dados e a dependência de uma classe externa indicam oportunidades de aprimoramento, como a inclusão de validações dinâmicas e suporte a pipelines.

5. Conclusão

A biblioteca de pré-processamento apresentada neste trabalho constitui uma solução eficaz, modular e extensível para o tratamento de dados tabulares em projeto de aprendizado de máquina. Desenvolvida inteiramente com recursos nativos da linguagem Python, oferece funcionalidades essenciais como tratamento de valores ausentes, escalonamento de variáveis numéricas e codificação de variáveis categóricas, além de validações estruturais que asseguram a integridade do dataset.

A arquitetura orientada a objetos, com separação clara de responsabilidades entre classes, favorece a reutilização, manutenção e evolução do código. A classe principal atua como interface unificadora, permitindo o encadeamento de operações e garantindo consistência na aplicação das transformações.

Os métodos implementados seguem, proporcionando flexibilidade na escolha de estratégias de imputação, escalonamento e codificação, conforme o contexto analítico e o tipo de variável.

Referências

[Menezes 2024]

MENEZES, N. N. C. Introdução à programação com Python. 4. ed. São Paulo: Novatec, 2024. ISBN 978-85-7522-886-9.

[Silva 2023]

SILVA, J. F. Pré-processamento de dados para aprendizado de máquina com Python. 1. ed. Rio de Janeiro: Ciência Moderna, 2023. ISBN 978-85-7393-999-1.